

Assessment 2: Individual experimental report including algorithm implementation

05/10/2026

100 Points Possible

Attempt 1



In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

▼ Details

Format: Report**Length:** 2000 words (+/-10%)**Weighting:** 40%[Download or print this page](#)[View rubric](#)

Overview

When writing a research paper, sometimes in addition to theoretical analysis, it is also important to include a section to demonstrate the performance in practice of the proposed algorithms (and data structures) by experiments. A good experiment section can often maximise the chance of a paper being accepted, since it serves as extra convincing evidence for the superiority of the proposition in the paper.

In this assessment, you are asked to conduct an experimental study on heap and write an experiment section to demonstrate the results as if you are writing a research paper on heap.

There are three main tasks in this assessment:

1. Implement a data generator, a binary heap (with a complete binary tree) and a simple competitor (an array)
2. Conduct the required experiments
3. Write a report to demonstrate and analyse the experimental results.

Purpose

This assessment develops practical your skills in implementing and evaluating data structures. You will bridge theory and practice by implementing a heap, conducting rigorous experiments, and analysing results scientifically. This mirrors real research methodology, teaching you how theoretical complexity analysis translates to actual performance and when different data structures excel.

Marking criteria

- **Criterion 1:** Experiment environment (5%)
- **Criterion 2:** Result demonstration (15%)
- **Criterion 3:** Analysis of experimental results (25%)
- **Criterion 4:** Conclusion and findings (30%)
- **Criterion 5:** Overall report quality and presentation (25%)

Assessment instructions

Complete each step to achieve the requirements of this assessment.

Step 1: Read through Task 1 – implement a data generator, binary heap and simple competitor.

Read through what your implemented data generator, binary heap and simple competitor need to achieve.

Note that there is no requirement on any specified programming language in this assessment – you can use whatever programming language you prefer. You may use AI tools (eg ChatGPT, GitHub Copilot) to assist with writing or debugging your code. However, you remain fully responsible for the final output, ensuring it meets all assessment requirements and that you understand the code you submit.

Data generator

In the experimental study, each data element is represented by an integer key, where key indicates the value of the element. The data generator should have four interfaces: `\text{gen\_element}()`, `\text{gen\_push}()`, `\text{gen\_pop}()` and `\text{getTop}()`.

Click each interface title to learn what these interfaces do.

`\text{gen\_element}()`

`\text{gen\_element}()` generates a new element by:

- `\text{key}` $\leftarrow$ an integer that is drawn uniformly at random from the range $[0, 10^7]$
- Return `\text{key}` as the generated element.

`\text{gen\_push}()`

`\text{gen\_push}()` is a push operation of an element in the form of `(1, \text{key})`, where `(1)` indicates that this operation is a push, and `\text{key}` is the element to be pushed.

Its detailed steps are:

- $\backslash(x \leftarrow)$ a new element of key generated by invoking $\backslash(\text{gen_element})()$
- return $\backslash((1, x))$, a push for element $\backslash(x)$

$\backslash(\text{gen_pop})()$

$\backslash(\text{gen_pop})()$ is a pop operation in the form of $\backslash(2)$, where $\backslash(2)$ indicates that this operation is a pop.

Its detailed steps are:

- Return $\backslash(2)$, a pop operation

$\backslash(\text{gen_getTop})()$

$\backslash(\text{gen_getTop})()$ is a getTop operation in the form of $\backslash(3)$, where $\backslash(3)$ indicates that this operation is a getTop.

Its detailed steps are:

- Return $\backslash(3)$, a getTop operation.

With the three interfaces $\backslash(\text{gen_push})()$, $\backslash(\text{gen_pop})()$ and $\backslash(\text{gen_getTop})()$ you can generate operation sequences with specified lengths. In particular, you can generate a file in the following format:

10

1 3

1 7

3

1 9

2

1 1

3

2

1 2

2

The number, 10, in the first line indicates the total number of operations in the sequence. Each of the subsequent $\backslash(10\backslash)$ lines indicates an operation. Specifically, the above file specifies a sequence of 10 operations.

Review the table, which specifies first a file that can be generated, and then what the sequence of operations are to generate this file.

File generated	Sequence of operations
10	N/A
1 3	$\backslash(\text{push}(3)\backslash)$: push an element with $\backslash(\text{key} = 3\backslash)$
1 7	$\backslash(\text{push}(7)\backslash)$
3	$\backslash(\text{getTop}()\backslash)$
1 9	$\backslash(\text{push}(9)\backslash)$

File generated	Sequence of operations
2	$\text{pop}()$
1 1	$\text{push}(1)$
3	$\text{pop}()$
2	$\text{getTop}()$
1 2	$\text{push}(2)$
2	$\text{pop}()$

The max-heap

Implement a max-heap with our complete binary tree version discussed in this subject. In your implementation, you can assume that the total number of elements in the heap is never more than (10^6) . Therefore, creating an array of length (10^6) suffices for all the experiments in this assessments.

The max-heap should be able to support the functionalities:

- $\text{push}(\text{key})$: push an element with key
- $\text{pop}()$
- $\text{getTop}()$
- $\text{heapify}()$

To implement these operations, you will need to implement $\text{bubble-up}(i)$ and $\text{bubble-down}(i)$, where (i) is the index of the node in the array, on which the operation is performed.

The competitor

The competitor in this experiment is an array (A) of length (10^6) . We assume the index of (A) starts from (0) . This array (A) maintains two information:

- a counter (cnt) which is initially (0) and is the current number of elements in (A) , and
- an index $(i_{\max})$ which is the index of the largest element in (A) ; if (A) is empty, i.e., $(\text{cnt} = 0)$, then $(i_{\max} = -1)$.

Moreover, (A) supports the operations $(\text{push}(\text{key}))$, $(\text{pop}())$ and $(\text{getTop}())$.

Click each operation title to learn what these interfaces do.

$(\text{push}(\text{key}))$

- $(A[\text{cnt}] \leftarrow \text{key})$, i.e., append (key) at the back of (A) ;
- if $(i_{\max} = -1)$ or $(A[i_{\max}] < A[\text{cnt}])$, $(i_{\max} \leftarrow \text{cnt})$;
- $(\text{cnt} \leftarrow \text{cnt} + 1)$;

$(\text{pop}())$

- if $(i_{\max} = -1)$, return (null) ;
- $(\text{key}_{\max} \leftarrow A[i_{\max}])$;
- swap element $(A[i_{\max}])$ with $(A[\text{cnt} - 1])$, the last element at the back of the array (A) ;
- delete $(A[\text{cnt} - 1])$ by setting $(\text{cnt} \leftarrow \text{cnt} - 1)$;

- if $(\text{cnt} = 0)$, set $(i_{\max} \leftarrow -1)$;
- find the new maximum element with the largest key by scanning the array (A) from index (0) to $(\text{cnt} - 1)$; let the index of this element be (i) ; update $(i_{\max} \leftarrow i)$;
- return $(\text{key}_{i_{\max}})$;

$(\text{getTop}())$

- if $(i_{\max} = -1)$, return (null) ;
- $(\text{key}_{i_{\max}} \leftarrow A[i_{\max}])$;
- return $(\text{key}_{i_{\max}})$;

Step 2: Read through Task 2 – experiments.

You will need to conduct four experiments for this assessment.

Click each heading to learn more about these experiments.

Experiment 1: Time vs length of push-only sequence

In this experiment, you study the total running time (of the max-heap and the above array (A) , respectively) vs the lengths (L) of sequences consisting of push operations only. Specifically, you first generate five push-only sequences $(\sigma_1, \sigma_2, \dots, \sigma_5)$ respectively with length $(L = 0.1M, 0.2M, 0.5M, 0.8M, 1M)$, where (M) stands for one million, i.e., (10^6) .

Then you input the same $(\sigma_1, \dots, \sigma_5)$ to each of the max-heap and the array (A) .

Experiment 2: Time vs getTop percentage

In this experiment, you will consider a sequence of a fixed length $(L = 1M)$ of operations (including both push and getTop). You should vary the in-expectation percentage, (getTop) , of getTop in five operation sequences $(\sigma_1, \dots, \sigma_5)$, where $(\text{getTop}) = 0.1\%, 0.5\%, 1\%, 5\%, 10\%$.

Specifically, an operation sequence (σ) with (getTop) is generated as follows:

- $(\sigma \leftarrow \emptyset)$;
- for $(i = 1)$ to $(L = 1M)$:
 - generate an operation (s_i) such that: (s_i) is a getTop with probability (getTop) , and it is a push with probability $(1 - \text{getTop})$;
 - add (s_i) to (σ) ;
- return the operation sequence (σ) ;

Run the max-heap and the array (A) on these sequences $(\sigma_1, \dots, \sigma_5)$, and measure their corresponding total running time.

Experiment 3: Time vs pop percentage

Analogous to experiment 2, you will consider a sequence of a fixed length $(L = 1M)$ of operations which contains push and pop only. You should vary the in-expectation percentage, (pop) , of the pop operations in the sequence, where $(\text{pop}) = 0.1\%, 0.5\%, 1\%, 5\%, 10\%$.

An operation sequence (σ) with (pop) is generated in an analogous way as the operation sequence with (getTop) in experiment 2. Run both of the max-

heap and the array A on these sequences and measure their corresponding overall running time on each of the sequences.

Experiment 4: Heapify vs push-one-by-one

In this experiment, you will reuse the push-only sequences $\sigma_1, \dots, \sigma_5$ in experiment 1. Compare the overall running time of the following two different construction methods for the max-heap on $(\sigma_1, \dots, \sigma_5)$.

Method 1: heapify(σ):

- initialise an empty max-heap H ;
- scan σ and collect all the elements to be pushed in the array of H ;
- heapify H to be a valid max-heap;
- return H ;

Method 2: push-one-by-one(σ)

- initialise an empty max-heap H ;
- invoke $H.\text{push}(\text{key})$ for each push operation in σ one by one;
- return H ;

Note: If an algorithm takes more than 3 hours in an experiment, then you may terminate it and record its running time as Out of Time .

Step 3: Read through Task 3 – the report.

You will need to write and submit a report on the experimental results. The report should contain the following components:

- **Experiment environment:** Explain clearly the necessary information for others to reproduce your experiments, such as: the CPU frequency, the size of memory, the operating system, the programming language, the compiler (if applicable), etc.
- **Experiments:**
 - **Result demonstration:** Report the results of the four experiments with diagrams. Specifically, in each diagram, the x-axis represents the variable: (L) in Exp 1, (getTop) in Exp 2, (pop) in Exp 3, and (L) in Exp 4. The y-axis is the corresponding overall running time of the max-heap and the array in Exp 1-3 and of the two construction methods in Exp 4. You will need to plot the results of the corresponding competitors. Hence, there will be two lines in each diagram.
 - **Analysis:** In addition to showing the experimental results, you are asked to analyse them. Specifically, you will need to explain, with certain theoretical analysis and complexity, why one algorithm or data structure performs better than the other in some cases while in other cases not. Sometimes, you will also need to analyse the results across different diagrams.
- **Conclusion:** Conclude your findings and observations in these experiments. For example, in which cases, which data structure or method is better.

Step 4: Work through the experiments, writing your report.

Execute all four experiments as described in Step 2. Record your results and write your report following the structure outlined in Step 3.

Step 5: Submit your report.

Ensure your report includes all required components:

- Experiment environment details
- All four experiment diagrams with results
- Comprehensive analysis of results with theoretical justification

- Conclusion summarising your findings.

Review your report for clarity, correctness and completeness before submission. Ensure all diagrams are clearly labelled and all code/implementations are included as appendices.

Assessment support

The referencing style for this subject is APA 7. Refer to the University's [Re:cite](https://library.unimelb.edu.au/recite) (<https://library.unimelb.edu.au/recite>) guide to assist with citation and reference list formatting.

Use the [Assessment 2 Q&A](#)  discussion board to ask questions about this assessment. For information about referencing styles, academic support and applying for extensions, visit the [Assessments](#)  page.

Submission checklist

Confirm you are ready to submit your assessment by checking the boxes.

I have:

Completed the four experiments

Detailed my experiment environment

Diagrammed my data series with correct axes labels

Reviewed my analysis to ensure it explains performance differences with theory

Identified in my conclusion which method performs better in each case

Assessed my work against the rubric criteria and made targeted improvements

Proofread my work to make sure it is free of spelling, grammar and punctuation errors

Viewed the [Assessment and results policy](#) to learn what is expected

University's [. \(https://policy.unimelb.edu.au/MPF1326\)](https://policy.unimelb.edu.au/MPF1326) of me to succeed.

Submit your assessment

View Rubric

Assessment 2: Written assessment				
Criteria	Ratings			
Experiment environment	H1 100%>80%	H2A 79%>75%	H2B 74%>70%	H3 69%>6
	All relevant environment details are reported clearly and completely. Information is presented in a manner that would allow full reproduction of the experiments by an independent researcher.	Nearly all environment details are present, with one minor detail missing or ambiguous. Reproduction would still be feasible despite the minor omission.	Most key environment details are present, but one or two moderately important details are absent. Reproduction would be possible but would require reasonable assumptions to fill gaps.	Some environ details are provided, but description is incomplete in that would ma reproduction difficult.
	5 to >3.99 pts	3.99 to >3.74 pts	3.74 to >3.49 pts	3.49 to >3.2

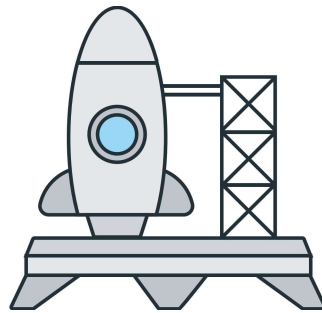
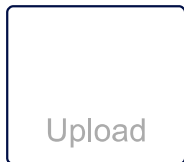
Criteria	Ratings			
Result demonstration	H1 100%>80%	H2A 79%>75%	H2B 74%>70%	H3 69%>6
	All four experiments are presented with diagrams. All diagrams are correctly and completely labelled. Both data series are plotted and clearly distinguishable.	All four experiments are presented with diagrams. Diagrams are mostly correctly labelled with only minor issues. Both data series are present but may lack clarity in distinguishing them.	All four experiments are present. Some diagrams have labelling errors or omissions. One data series may be missing or unclear in one diagram.	Three of the four experiments are presented with diagrams, or are present but have notable labelling errors across multiple diagrams. A data series may be missing or unclear across multiple diagrams.
	15 to >11.99 pts	11.99 to >11.24 pts	11.24 to >10.49 pts	10.49 to >9.74 pts
Analysis of experimental results	H1 100%>80%	H2A 79%>75%	H2B 74%>70%	H3 69%>6
	Rigorous theoretical and complexity analysis is used to explain the observed results across all four experiments. Cross-experiment comparisons are made where relevant.	Theoretical and complexity analysis is used to explain the observed results across most experiments, with minor gaps. Cross-experiment comparisons are made but may lack depth.	Theoretical and complexity analysis is used to explain the observed results, but reasoning is superficial in places, or one experiment is not addressed. Cross-experiment comparisons are attempted but incomplete.	Some theoretical and complexity analysis is present but is inconsistently applied across experiments. Cross-experiment comparisons are absent or assessed without explanation.
	25 to >19.99 pts	19.99 to >18.74 pts	18.74 to >17.49 pts	17.49 to >16.24 pts

Criteria	Ratings			
Conclusion and findings	H1 100%>80%	H2A 79%>75%	H2B 74%>70%	H3 69%>6
	Conclusions are correct and specific about when each data structure or method is preferable.	Conclusions are correct and mostly specific but may lack nuance or specificity in one area.	Conclusions are broadly correct but lack specificity in places, or one experiment is not addressed.	Conclusions are present but are incomplete or contain minor inaccuracies.
	30 to >23.99 pts	23.99 to >22.49 pts	22.49 to >20.99 pts	20.99 to >19.4
Overall report quality and presentation	H1 100%>80%	H2A 79%>75%	H2B 74%>70%	H3 69%>6
	The report is well-structured with clearly delineated sections. Writing is precise, professional, and free of grammatical or spelling errors. The word count is within the required limit, and all code is included as an appendix.	The report is well-structured with clearly delineated sections. Writing is clear with only minor grammatical or spelling errors. The word count is within the required limit, and code is mostly included as an appendix.	The report has a clear structure, but some sections are underdeveloped. Writing is generally readable but contains noticeable errors. The word count is close to the required limit, but code may not be fully included as an appendix.	The report structure is identifiable but inconsistent. It contains frequent errors that impact clarity in places. The word count may be outside the required limit, and code is not included as an appendix.
	25 to >19.99 pts	19.99 to >18.74 pts	18.74 to >17.49 pts	17.49 to >16.4

Academic integrity declaration

By submitting work for assessment I hereby declare that I understand the University's policy on academic integrity and statement on the use of artificial intelligence software.

In accordance with these documents, I declare that the work submitted is original and solely my work, and that I have not been assisted by another person (collusion) apart from where the submitted work is for a designated collaborative task, in which case the individual contributions are indicated. I also declare that I have not used any editing tools or sources without proper acknowledgment (plagiarism). Where the submitted work is a computer program or code, I further declare that any copied code is declared in comments identifying the source at the start of the program or in a header file, that comments inline identify the start and end of the copied code, and that any modifications to code sources elsewhere are commented upon as to the nature of the modification.

Choose a submission type.

Choose a file to upload

or

 Webcam Photo

 Canvas Files

☐ I agree to the tool's **[End-User License Agreement](https://api.turnitin.com/api/lti/1p0/user/static_eula)**
(https://api.turnitin.com/api/lti/1p0/user/static_eula)

This assignment submission is my own, original work

*

Submit Assignment